

Document Type: Technical Infrastructure Specification

Author: CTO & Principal DevOps Architect

Target Version: v1.0.0

Last Updated: July 2026

1. Executive Summary

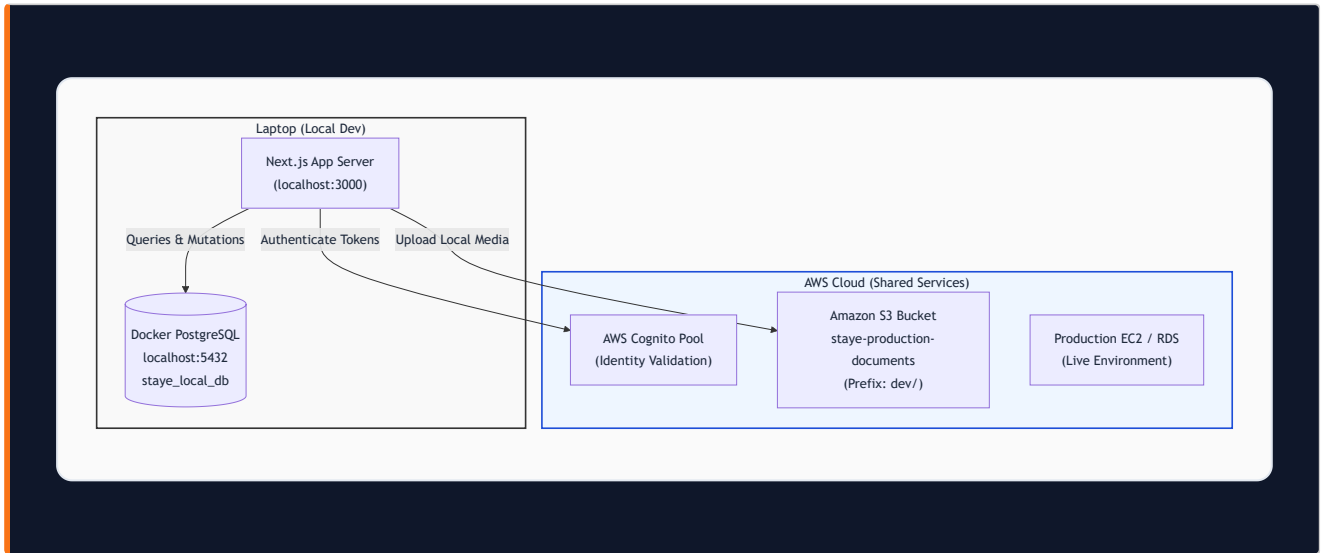
This specification defines the complete local development architecture for **Stayee Anywhere**. The primary mandate is **100% data and environment isolation**: developers work on local feature branches using a local Docker PostgreSQL database populated from a cloned production snapshot.

Key Architectural Pillars

- **Zero Risk to Production:** Local development never executes write queries against the live AWS RDS database.
- **S3 Namespace Isolation:** All local document uploads use `AWS_S3_KEY_PREFIX="dev/"` to prevent cluttering production customer documents.
- **Identity-First Authentication:** Local NextAuth connects seamlessly to real AWS Cognito User Pools for user authentication.
- **Strict Deployment Boundaries:** Only pull requests merged into `main` trigger AWS EC2 deployment via GitHub Actions.

2. System Architecture & Component Mapping

The system splits components between local developer hardware and shared AWS cloud services:



Component Isolation Matrix

APP SERVER	Laptop (localhost:3000)	npm run dev	Feature branches isolated
DATABASE	Docker (localhost:5432)	stayer_local_db	100% Local Container
SSM RDS TUNNEL	Laptop (localhost:5433)	Port Forwarding to RDS	Dump-only connection
AWS COGNITO	AWS Cloud (ap-south-1)	Shared User Pool ID	Validates tokens
AWS S3 BUCKET	AWS Cloud (ap-south-1)	dev/ Prefix	Production documents untouched

3. Environment Configuration Guide (`.env.local`)

Below is the complete, line-by-line `.env.local` template required for local development:

```
# — LOCAL DOCKER DATABASE (Isolated) —
DATABASE_URL="postgresql://postgres:password@localhost:5432/stage_local_db"
DIRECT_URL="postgresql://postgres:password@localhost:5432/stage_local_db"
DB_IS_TUNNEL="false"

# — S3 DEV ISOLATION PREFIX —
AWS_S3_KEY_PREFIX="dev/"
AWS_S3_BUCKET_NAME="stage-production-documents"
AWS_REGION="ap-south-1"

# — AWS COGNITO CREDENTIALS —
COGNITO_CLIENT_ID="69hickkdp3cj5qekr8jbqjnnpm"
COGNITO_CLIENT_SECRET="1qjbnf2puajqghrivs6de3bbogqsq5qtl6cbajvg55ksujppfda"
COGNITO_ISSUER="https://cognito-idp.ap-south-1.amazonaws.com/ap-south-1_SK2jWUrow"
NEXTAUTH_SECRET="stage_super_secret_dev_key_2026"
NODE_ENV="development"
```

[!IMPORTANT] Never commit `.env.local` to version control. `.env.local` is listed in `.gitignore` to keep local secrets secure.

4. One-Time Setup: Database Cloning & Restoration

Step 1: Start Local Docker PostgreSQL

Ensure Docker Desktop is running, then execute:

```
docker-compose up -d
```

Step 2: Open SSM Tunnel to Production RDS (Port 5433)

To avoid port conflicts with local Docker on `5432`, forward the SSM tunnel to port `5433`:

```
$INSTANCE_ID = "i-066e2193c831d8495"
$RDS_HOST = "database-1.cj2woqyom1ds.ap-south-1.rds.amazonaws.com"

& "C:\Program Files\Amazon\AWSCLIV2\aws.exe" ssm start-session `
  --target $INSTANCE_ID `
  --document-name AWS-StartPortForwardingSessionToRemoteHost `
  --parameters "{\"host\":\"$RDS_HOST\"\",\"portNumber\":\"5432\"\",\"localPortNumber\":
[\"5433\"]}" `
  --region ap-south-1
```

Step 3: Export Production Database Dump

In a second PowerShell window, run:

```
pg_dump --host=localhost --port=5433 --username=postgres --dbname=staye_db --no-owner --no-acl --
format=plain --file=prod_dump.sql
```

Step 4: Restore Snapshot into Local Docker Database

Import the snapshot into your local container:

```
psql --host=localhost --port=5432 --username=postgres --dbname=staye_local_db --
file=prod_dump.sql
```

Step 5: Clean Up Dump File & Close Tunnel

```
Remove-Item prod_dump.sql
# Stop the SSM tunnel window (Ctrl+C)
```

5. Prisma Migration Synchronization Protocol

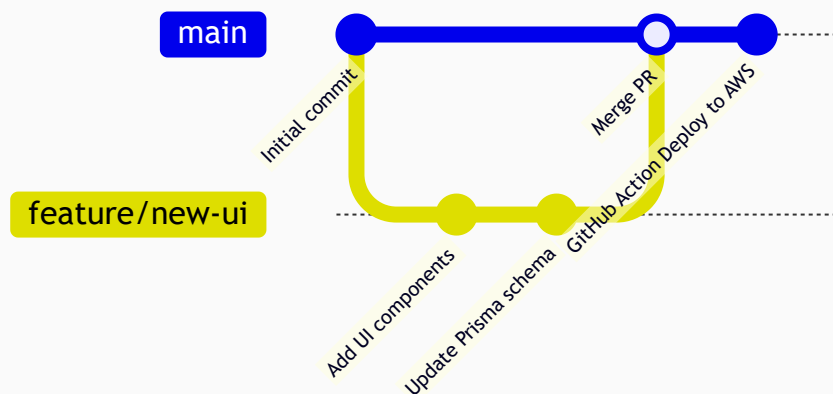
After restoring a database dump, mark all existing Prisma migrations as applied:

```
npx prisma migrate resolve --applied "20260619092505_init"
npx prisma migrate resolve --applied "20260621154506_add_hostel_affidavit_text"
npx prisma migrate resolve --applied "20260627182323_food_dashboard_v2"
npx prisma migrate resolve --applied "20260709000000_add_task_assignment_module"
npx prisma migrate resolve --applied "20260709074717_add_task_assignment_module"
npx prisma migrate resolve --applied "20260721090000_rename_cognito_sub_add_hashed_password"
```

Verify migration status:

```
npx prisma migrate status
```

6. Day-to-Day Developer Workflow & CI/CD Guardrails



1. Feature Branch Creation:

```
git checkout main
git pull origin main
git checkout -b feature/your-feature-name
```

2. **Schema Mutations:** If modifying `schema.prisma`, generate a migration:

```
npx prisma migrate dev --name feature_description
```

3. **Local Build Check:** Always verify before opening a PR:

```
npm run build
```

4. **Deploying to Production:** Merge your PR into `main`. GitHub Actions automatically builds the Docker container, pushes to ECR, runs `prisma migrate deploy` on production RDS, and restarts the EC2 app instance.

7. Verification Checklist

- ☐ Docker container `staye_local_db` is running on `localhost:5432`.
- ☐ `.env.local` contains `DB_IS_TUNNEL="false"` and `AWS_S3_KEY_PREFIX="dev/"`.
- ☐ `npm run build` completes with 0 errors.
- ☐ Application loads at `http://localhost:3000` with admin credentials (`admin@nexthome.io` / `Password@123`).